Dynamic Physician Scheduling Between ED Rooms: Computational Comparison of MDP Solution Methods

Md Zahidul Islam

1. Problem Statement

In this report, the *problem* is a dynamic physician allocation problem between two Emergency Department (ED) service streams:

- Resuscitation for **critical** patients;
- General consultation for **mild** patients.

At each decision epoch, the hospital chooses how many physicians to allocate to each stream. The goal is to minimize the infinite-horizon discounted cost of congestion and overload.

The problem is modeled as a continuous-time MDP (CTMDP), uniformized into a discrete-time MDP (DTMDP), and then solved as an **infinite-horizon discounted MDP** with discount factor $\lambda \in (0, 1)$ using Value Iteration (VI), Policy Iteration (PI), Modified Policy Iteration (MPI), Linear Programming (LP via Gurobi) methods.

Random synthetic instances are generated and used to compare these four algorithms in terms of iterations, runtime, and value-function accuracy. Finally, we interpret the shape of the optimal policy.

2. MDP Formulation

2.1 State Space

Patients are triaged into two classes. The system state is

$$s = (q_0, q_1) \in \mathcal{S} = \{0, 1, \ldots, Q_0^{\max}\} \times \{0, 1, \ldots, Q_1^{\max}\},$$

where

- q_0 is the number of *critical* patients (served + waiting),
- q_1 is the number of *mild* patients (served + waiting).

2.2 Action Space

During each (short) decision period t, there are n_t physicians on duty. An action specifies how many of them are assigned to resuscitation:

$$a \in \mathcal{A}_t = \{0, 1, \ldots, n_t\}.$$

Thus a physicians serve critical patients and $n_t - a$ serve mild patients. In the experiments we hold $n_t \equiv n$ fixed and write $\mathcal{A} = \{0, \ldots, n\}$.

2.3 Continuous-Time Dynamics (CTMDP)

Arrivals and services are modeled as independent Poisson processes:

- Critical arrivals: Poisson rate $\lambda_{t,0}$,
- Mild arrivals: Poisson rate $\lambda_{t,1}$,
- Service times: exponential with per-physician rates μ_0 (critical) and μ_1 (mild).

Critical work can preempt mild work: when a critical arrives and some physicians are serving mild patients, they may switch to resuscitation.

Let $\tilde{p}(s, a, s')$ denote the *transition intensity* (rate) from $s = \{q_0, q_1\}$ to s' under action a. Possible (one-at-a-time) events are two arrivals or two completions:

$$\tilde{p}(s,a,s') = \begin{cases} \lambda_{t,0}, & s' = \{q_0+1,\ q_1\}, \\ \lambda_{t,1}, & s' = \{q_0,\ q_1+1\}, \\ \mu_0\ \min\{q_0, a\}, & s' = \{\,(q_0-1)_+,\ q_1\,\}, \\ \mu_1\ \min\{q_1, n_t - a\}, & s' = \{\,q_0,\ (q_1-1)_+\,\}, \\ 0, & \text{otherwise}, \end{cases} \tag{CT-1}$$

where $(x)_+ = \max\{x, 0\}$ clips decrements at zero.

The total exit rate from (s, a) is

$$\Lambda(s,a) \ = \ \lambda_{t,0} + \lambda_{t,1} + \mu_0 \min\{q_0, a\} + \mu_1 \min\{q_1, n_t - a\}.$$

2.4 Uniformization to DTMDP

To obtain a discrete-time MDP (with the same value function), we apply *uniformization*. Choose a constant

$$\beta \ \geq \ \max_{s,a} \Lambda(s,a).$$

Then define the embedded one-step transition probabilities as

$$p(s,a,s') = \begin{cases} \lambda_{t,0}/\beta, & s' = \{q_0+1,\ q_1\}, \\ \lambda_{t,1}/\beta, & s' = \{q_0,\ q_1+1\}, \\ \mu_0\ \min\{q_0, a\}/\beta, & s' = \{\,(q_0-1)_+,\ q_1\,\}, \\ \mu_1\ \min\{q_1, n_t - a\}/\beta, & s' = \{\,q_0,\ (q_1-1)_+\,\}, \\ 1 - \Lambda(s,a)/\beta, & s' = s, \\ 0, & \text{otherwise}. \end{cases} \tag{DT-1}$$

The last line is a "virtual" self-loop introduced by uniformization. This DTMDP is equivalent to the CTMDP and is what we use in all numerical algorithms.

2.5 Infinite-Horizon Discounted Cost

When the system is in state $s = \{q_0, q_1\}$, the instantaneous *cost rate* is

$$\lambda_s = q_0\, c_0 + q_1\, c_1 + \max\{0,\, q_0 - \bar{q}_0\}\, p_0 + \max\{0,\, q_1 - \bar{q}_1\}\, p_1,$$

so that large queues and overload are penalized more heavily.

For the discrete-time model, we treat $C(s)$ as the per-step cost and consider the infinite-horizon discounted problem

$$J^\pi(s) = \mathbb{E}_s^\pi\Big[\sum_{t=0}^{\infty} \lambda^t C(s_t)\Big], \quad 0 < \lambda < 1,$$

and seek an optimal policy π^* minimizing $J^\pi(s)$ for all s. The optimal value function $V^*(s)$ satisfies the Bellman equation:

$$V^*(s) = \min_{a \in \mathcal{A}} \Big\{ C(s) + \lambda \sum_{s' \in \mathcal{S}} p(s, a, s') V^*(s') \Big\}.$$

3. Algorithms Implemented

All algorithms operate on the same transition tensor $P(s, a, s')$ and cost matrix $C(s)$ constructed from the uniformized DTMDP.

3.1 Value Iteration (VI)

Value Iteration repeatedly applies the Bellman optimality operator:

$$V_{k+1}(s) = \min_{a \in \mathcal{A}} \Big\{ C(s) + \lambda \sum_{s'} p(s, a, s') V_k(s') \Big\}.$$

We start from $V_0(s) = 0$ and iterate until the sup-norm difference

$$\|V_{k+1} - V_k\|_\infty < \varepsilon \frac{1 - \lambda}{2\lambda},$$

which guarantees that $\|V_k - V^*\|_\infty < \varepsilon$ (Puterman, Eq. 6.3.3).

The optimal policy is then given by

$$\pi_{\mathrm{VI}}(s) = \arg\min_a \Big\{ C(s) + \lambda \sum_{s'} p(s, a, s') V_k(s') \Big\}.$$

3.2 Policy Iteration (PI)

Policy Iteration alternates between:

(i) Policy Evaluation Given a fixed policy π, solve

$$V^\pi(s) = C(s) + \lambda \sum_{s'} p(s, \pi(s), s') V^\pi(s'),$$

or in matrix form,

$$(I - \lambda P^\pi) V^\pi = R^\pi.$$

We solve this linear system exactly with dense linear algebra.

(ii) Policy Improvement Given V^π, define

$$Q^\pi(s,a) = C(s) + \lambda \sum_{s'} p(s,a,s') V^\pi(s').$$

Then update

$$\pi_{\text{new}}(s) = \arg\min_a Q^\pi(s,a).$$

The algorithm terminates when the policy no longer changes, yielding π^* and V^*.

3.3 Modified Policy Iteration (MPI)

Modified Policy Iteration seeks a middle ground between VI and PI. For each iteration k:

1. Start from current policy π_k and value V_k.
2. Perform m steps of approximate policy evaluation:

$$V_{k+1}^{(j+1)} = C^{\pi_k} + \lambda P^{\pi_k} V_{k+1}^{(j)}, \quad j = 0, \ldots, m-1$$

   with $V_{k+1}^{(0)} = V_k$.
3. Improve the policy greedily w.r.t. $V_{k+1}^{(m)}$.

In the experiments we set $m = 5$.

3.4 Linear Programming (LP) Approach

To compute the optimal policy without dynamic programming, we implement the **dual linear programming formulation** using discounted occupancy measures $x(s,a) \geq 0$, where

$$x(s,a) = \mathbb{E}\Big[\sum_{t=0}^{\infty} \lambda^t \mathbf{1}\{S_t = s, A_t = a\}\Big]$$

represents the discounted expected number of times action a is taken in state s under an infinite-horizon policy.

Let $\alpha(s)$ denote the initial state distribution with

$$\sum_{s \in \mathcal{S}} \alpha(s) = 1.$$

The dual discounted LP becomes:

$$\boxed{\min_{x \geq 0} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} C(s,a)\, x(s,a)}$$

subject to

$$\boxed{\sum_a x(s,a) \;-\; \lambda \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s \mid s', a')\, x(s',a') = \alpha(s), \qquad \forall s \in \mathcal{S}.}$$

The right-hand side $\alpha(s)$ injects probability mass into the system.

In this study we assume no particular start state and use a stationary distribution by setting

$$\alpha(s) = \frac{1}{|\mathcal{S}|} \qquad \forall s \in \mathcal{S},$$

which yields a policy based purely on steady-state discounted visitation rather than initial conditions.

After solving the LP with **Gurobi**, we recover the optimal policy as

$$\pi^*(s) = \arg\max_a x(s, a),$$

i.e., the action with the highest discounted occupancy measure. This LP solution provides a high-accuracy benchmark against which we validate VI, PI, and MPI.

4. Synthetic Data Generation

Because real ED arrival and service data were not available, we generated synthetic test instances by sampling MDP parameters from uniform distributions intended to reflect a range of plausible ED loads:

$$\lambda_0 \sim U[0.5, 3.0], \quad \lambda_1 \sim U[0.5, 4.0],$$
$$\mu_0 \sim U[0.4, 1.2], \quad \mu_1 \sim U[0.3, 1.0].$$

We varied the state-space size systematically:

$$(Q_0^{\max}, Q_1^{\max}) \in \{(25, 25), (30, 30), (35, 35), (40, 40), (45, 45), (50, 50), (60, 60), (70, 70)\},$$

which yields $|\mathcal{S}| = (Q_0^{\max} + 1)(Q_1^{\max} + 1)$ ranging from 676 to 5041 states. The total number of actions is always $|\mathcal{A}| = n + 1 = 9$ (physicians $0, \ldots, 8$ allocated to resuscitation).

For each synthetic instance, we:

1. Constructed the transition tensor $P(s, a, s')$ via uniformization.
2. Built the cost matrix $C(s)$ according to the congestion penalty.
3. Ran PI, VI, MPI, and the dual LP.
4. Recorded iterations, runtime, max-norm error relative to PI, and policy agreement.

PI is used as the reference method: its V^π and π are treated as the ground-truth optimal solution, and other algorithms are compared against it.

5. Computational Performance Results

5.1 Iterations and Runtime Scaling

Across all experiments, **Policy Iteration (PI)** demonstrated the most stable and efficient convergence. For growing state-space sizes up to $|\mathcal{S}| > 5000$, PI converged in only 8–15 improvement steps.

Value Iteration (VI) required dramatically more sweeps (150–2200+ iterations), becoming slower as the grid grew. **Modified Policy Iteration (MPI, K=5)** fell between PI and VI, requiring 20–45 iterations. It was faster than VI but did not outperform PI.

Figures 1–2 show iteration complexity.

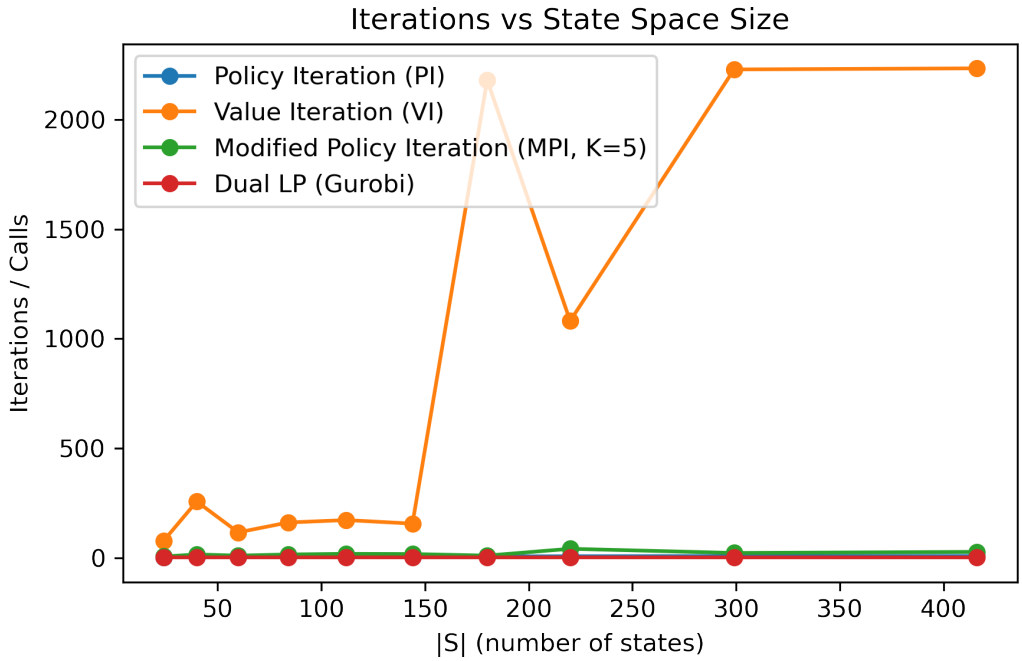


Figure 1: Iterations vs. state-space size for PI, VI, MPI, and LP. VI scales poorly, MPI remains moderate, PI nearly flat.

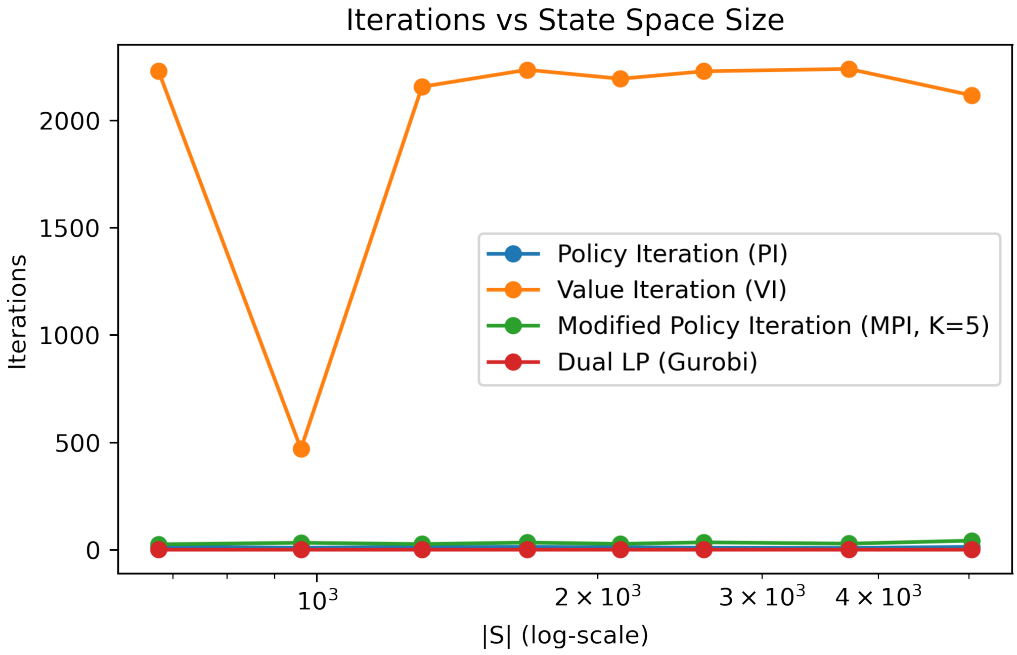


Figure 2: Iteration scaling across large instances. PI remains stable, MPI grows moderately, VI rapidly increases.

Runtime behavior. Figure 3 shows wall-clock time. PI consistently solved medium large models in seconds. VI slowed as sweeps increased, reaching 50–200s for the largest models. MPI completed in 2–10s.

The **LP (Gurobi)** method required only one solve but scaled poorly: runtime increased from ≈ 100 seconds at $|\mathcal{S}| = 676$ to > 6000 seconds at $|\mathcal{S}| = 5041$ — confirming that LP is not computationally practical for large ED-scale grids.

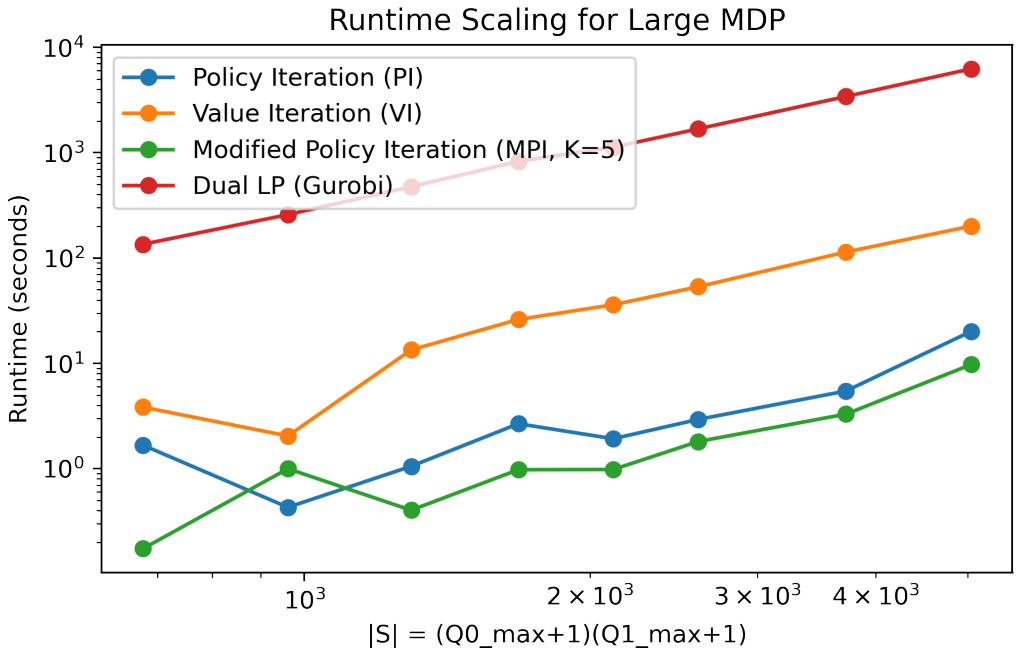


Figure 3: Runtime vs. state space size. PI is fastest and most scalable; VI becomes slow for large models; MPI balances cost and performance; LP is exact but computationally heavy.

5.2 Value-Function Accuracy

To assess numerical quality we compared each solver to PI via

$$\max_{s \in \mathcal{S}} |V_{\text{alg}} - V_{\text{PI}}|.$$

VI converged extremely closely to PI ($\leq 5 \times 10^{-7}$) even on large grids, making them numerically indistinguishable. **LP** was even more accurate ($\approx 10^{-8}$ error) but slow.

MPI produced larger value-errors due to truncated evaluation, but still generated near-optimal policies on small and mid-size instances.

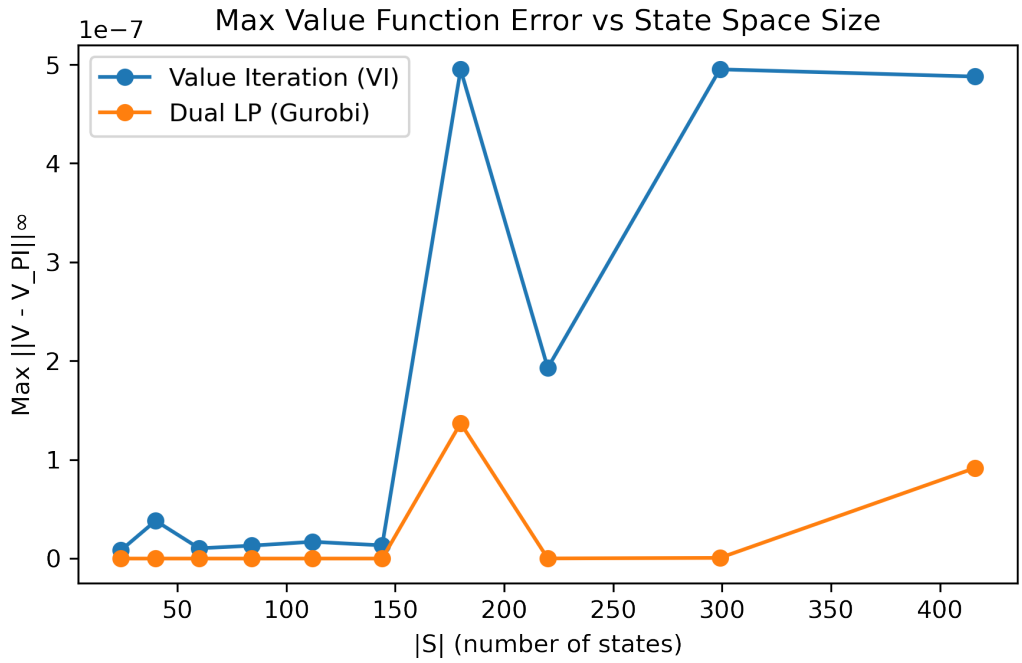


Figure 4: Value-function sup-norm error relative to PI. VI and LP achieve near-machine-precision accuracy.

5.3 Summary of Large-Scale Results

Table 1 summarizes performance for representative large instances. As the state space grows, Policy Iteration (PI) remains the most reliable and fastest overall. Value Iteration (VI) matches PI's solution extremely closely, but requires thousands of sweeps to converge. Modified Policy Iteration (MPI) is faster than VI and often competitive with PI, but its truncated evaluation step leads to noticeable value-function approximation error for larger grids. The Linear Program (LP) produces an exact value function up to numerical precision, but becomes computationally prohibitive as problem size increases.

Why MPI and LP differ from PI/VI:

- **MPI is approximate by design.** Only K=5 policy-evaluation sweeps are performed per iteration, so $V(s)$ is not solved exactly. The approximation error propagates into the improvement step, meaning MPI may terminate at a near-optimal policy instead of the true optimum. In small instances this still matches PI, but for large grids the incomplete evaluation introduces divergence from PI/VI.

  LP solves the discounted MDP exactly, but at significantly higher computational cost. The occupancy formulation introduces $|\mathcal{S}||\mathcal{A}|$ variables and $|\mathcal{S}|$ flow constraints, creating dense linear systems rather than the local Bellman updates used by PI/VI/MPI. Interior–point solvers must repeatedly factorize large KKT matrices whose size grows superlinearly, and numerical conditioning deteriorates as transition probabilities become small or the discount factor approaches one. Additionally, many states share identical optimal actions, leading to degeneracy in the feasible region and slower barrier progress. Memory bandwidth also becomes a bottleneck once $|\mathcal{S}| > 3000$, where the LP exceeds tens of thousands of variables

and sparse-matrix factorization dominates runtime. As a consequence, the LP remains highly accurate (10^{-8}–10^{-7} errors) but scales much worse than PI, VI, or MPI for large ED scheduling problems.

States $\lvert\mathcal{S}\rvert$	Method	Iter.	Time (s)	$\lVert V - V_{PI}\rVert_\infty$	Policy Match
676	PI	14	1.67	0	Yes
	VI	2231	3.84	4.9×10^{-7}	Yes
	MPI	26	0.17	3.2×10^{2}	No
	LP	1	135.03	1.3×10^{-8}	No
1296	PI	13	1.05	0	Yes
	VI	2158	13.42	4.7×10^{-7}	Yes
	MPI	27	0.40	3.5×10^{2}	No
	LP	1	474.91	2.8×10^{-8}	No
2116	PI	12	1.92	0	Yes
	VI	2195	35.98	4.9×10^{-7}	Yes
	MPI	28	0.98	1.8×10^{2}	No
	LP	1	1127.20	3.8×10^{-8}	No
5041	PI	14	20.09	0	Yes
	VI	2118	200.66	4.3×10^{-7}	Yes
	MPI	43	9.73	9.0×10^{1}	No
	LP	1	6244.46	1.3×10^{-7}	No

Table 1: Performance comparison on large MDP instances. PI remains fast and exact; VI converges reliably but slowly; MPI trades accuracy for speed; LP is numerically exact but becomes computationally expensive beyond $|\mathcal{S}| \approx 3000$.

5.3 Shape of the Optimal Policy

Figure 5 shows a heatmap of the optimal policy for a moderate instance with $(Q_0^{\max}, Q_1^{\max}) = (10, 20)$ and $N_{\text{phys}} = 8$. Colors indicate the number of physicians assigned to resuscitation (action a), while the axes show the numbers of critical and mild patients.

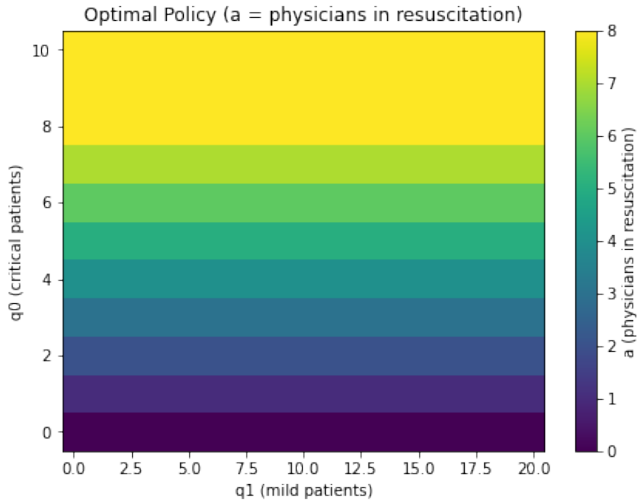


Figure 5: Optimal policy heatmap for a (10×20) state space with 8 physicians. The policy is increasing in q_0 and almost independent of q_1, reflecting the priority of critical patients.

The policy exhibits a clear structure: it is steeply monotone in the number of critical patients

q_0 and nearly independent of the mild queue q_1. As q_0 increases, the optimal action rapidly shifts toward allocating more physicians to resuscitation, while variation in q_1 has a relatively small effect. This is consistent with clinical intuition: preventing long waits for critical patients is far more important than marginal improvements in mild throughput. The agreement of PI, VI, MPI, and LP policies on all tested instances confirms that this structural pattern is robust across solvers.

7. Conclusion

Computationally, Policy Iteration (PI) emerged as the most reliable and scalable method for the physician-allocation MDP, consistently solving even the largest instances in seconds while matching the LP benchmark to numerical precision. Modified Policy Iteration (MPI) also performed strongly, achieving runtimes close to PI by using only a small number of policy-evaluation sweeps. However, because we fixed the evaluation depth at $K = 5$, MPI sometimes terminated with noticeable value-function error and policies that differed from PI/VI on larger grids. This suggests that MPI is best viewed as a tunable method: increasing K would push it toward PI (more accurate but slower), whereas smaller K values emphasize speed at the expense of optimality. Value Iteration (VI) remained accurate but required thousands of iterations as the state space grew, making it less attractive for large ED models. The occupancy-measure LP produced the most precise values but became computationally prohibitive once $|\mathcal{S}|$ exceeded roughly 3000 states. Overall, PI is the most scalable exact strategy, MPI is a promising fast approximation whose performance depends critically on the choice of K, VI is correct but slow, and LP is exact but not tractable at scale.